Lesson:

MultiThreading

List of Concepts Involved:

- What is an Operating System?
- Need of Multiple thread
- What is Thread?
- How to create Threads
- run() method
- Multiple task within single run()
- Different states of Thread
- synchronised keyword
- Deadlock
- Producer - Consumer problem

What is an Operating System?

It is a system software which runs in the background and helps the user to run other applications.

MultiTasking

Executing several tasks simultaneously is the concept of multitasking.
There are 2 types of Multitasking.
a. Process based multitasking
b. Thread based multitasking.

Process based multitasking

Executing several tasks simultaneously where each task is a separate independent process such type of multitasking is called "**process based multitasking**".
Example
typing a java program
listening to a song
downloading the file from internet
Process based multitasking is best suited at "os level".

Thread based multitasking

Executing several tasks simultaneously where each task is a separate independent part of the same Program, is called "Thread based MultiTasking".
Each independent part is called a "Thread".

1. This type of multitasking is best suited at "Programmatic level".
The main advantages of multitasking is to reduce the response time of the system and to improve the performance.

2. The main important application areas of multithreading are
   a. To implement multimedia graphics
   b. To develop web application servers
   c. To develop video games

3. Java provides inbuilt support to work with threads through API called
   Thread,Runnable,ThreadGroup,ThreadLocal,...

4. To work with multithreading, java developers will code only for 10% remaining
   90% java API will take care..

What is thread?

A. Separate flow of execution is called "Thread".
if there is only one flow then it is called "SingleThread" programming.
For every thread there would be a seperate job.

B. In java we can define a thread in 2 ways
a. implementing Runnable interface
b. extending Thread class

ThreadScheduler

If multiple threads are waiting to execute,then which thread will execute 1st is decided by ThreadScheduler which is part of JVM.
In the case of MultiThreading we can't predict the exact output, only possible output we can expect.
Since jobs of threads are important,we are not interested in the order of execution; it should just execute such that performance should be improved.

diff b/w t.start() and t.run()

if we call t.start() and seperate thread will be created which is responsible to execute the run() method.
if we call t.run(), no separate thread will be created, rather the method will be called just like a normal method by main thread.

Importance of Thread class start() method

For every thread, required mandatory activities like registering the thread with Thread Scheduler will be taken care by Thread class
start() method and the programmer is responsible for just doing the job of the Thread inside run() method.
start() acts like an assistance to programmers.

```
 start()
    {
register thread with ThreadScheduler
           All other mandatory low level activities
 invoke or call the run() method.
    }
```

We can conclude that without executing the Thread class start() method there is no chance of starting a new Thread in java.
Due to this, start() is considered the **heart of MultiThreading.**

If we are not overriding run() method

If we are not Overriding run() method then Thread class run() method will be executed which has an empty implementation and hence we won't get any output.

eg::

```
class MyThread extends Thread{}
class ThreadDemo{
 public static void main(String... args){
  MyThread t=new MyThread();
  t.start();
 }
}
```

Overloading of run() method

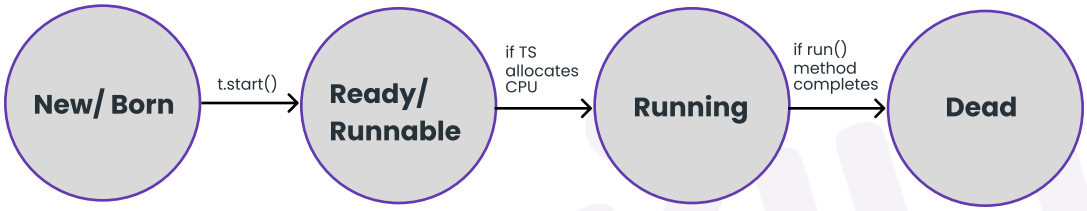
We can overload the run() method but Thread class start() will always call run() with zero argument.
if we overload run method with arguments, then we need to explicitly call argument based run method and it will be executed just like normal method.

Life cycle of a Thread

- if Thread scheduler allocates CPU time then we say thread entered into Running state.
- if run() is completed by thread then we say thread entered into dead state.
- Once we create a Thread object then the Thread is said to be in a new state or born state.
- Once we call start() method then the Thread will be entered into Ready or Runnable state.
- If Thread Scheduler allocates CPU then the Thread will be entered into running state.
- Once the run() method completes then the Thread will enter into dead state.

Life Cycle Of Thred

MyThread t=new My Thread();



Different approach for creating a Thread?

A. extending Thread class
B. implementing Runnable interface

Which approach is the best approach?

- implements Runnable interface is recommended becoz our class can extend another class through which inheritance benefit can be brought into our class.
- Internally performance and memory level is also good when we work with interfaces.
- if we work with extended features then we will miss out inheritance benefit because already our class has inherited the feature from "Thread class", so we normally don't prefere extended approach rather implements approach is used in real time for working with "MultiThreading".

Various Constructors available in Thread class

Thread t=new Thread()
Thread t=new Thread(Runnable r)
Thread t=new Thread(String name)

Names of the Thread

Internally for every thread,there would be a name for the thread.
a. name given by jvm
b. name given by the user.

ThreadPriorities

- For every Thread in java has some priority.
- The valid range of priority is 1 to 10,it is not 0 to 10.
- if we try to give a different value then it would result in "IllegalArgumentException".
- Thread.MIN_PRIORITY = 1
- Thread.MAX_PRIORITY = 10
- Thread.NORM_PRIORITY = 5
- If both the threads have the same priority then which thread will get a chance as a program we can't predict becoz it is vendor dependent.

We can set and get priority values of the thread using the following methods
a. public final void setPriority(int priorityNumber)
b. public final int getPriority()
The allowed priorityNumber is from 1 to 10,if we try to give other values it would result in **"IllegalArgumentException".**
System.out.println(Thread.currentThread().
setPriority(100);//IllegalArgumentException.

DefaultPriority

The default priority for only the main thread is "5",whereas for other threads priority will be inherited from parent to child.
Parent Thread priority will be given as Child Thread Priority.

We can prevent Threads from Execution
a. yield()
b. sleep()
c. join()

yield()

- It causes the current executing Thread to give a chance for waiting Threads of the same priority.
- If there is no waiting Threads or all waiting Threads have low priority then

the same Thread can continue its execution.

- If all the threads have the same priority and if they are waiting then which thread will get a chance we can't expect, it depends on ThreadScheduler.
- The Thread which is yielded, when it will get the chance once again depends on the mercy of "ThreadScheduler" and we can't expect exactly.

public static native void yield()

join()

If the thread has to wait until the other thread finishes its execution then we need to go for join().
if t1 executes t2.join() then t1 should wait till t2 finishes its execution.
t1 will be entered into waiting state until t2 completes, once t2 completes then t1 can continue with its execution.

eg#1.

venue fixing ======> t1.start()
wedding card printing ======> t2.start()======> t1.join()
wedding card distribution ======> t3.start()======> t2.join()

Waiting of Child Thread until Completing Main Thread

We can make the main thread wait for the child thread as well as we can make the child thread also wait for the main thread.

sleep()

If a thread doesn't want to perform any operation for a particular amount of time then we should go to sleep().

Signature

```
public static native void sleep(long ms) throws InterruptedException
public static void sleep(long ms,int ns) throws InterruptedException
```

Every sleep method throws InterruptedException, which is a checked exception so we should compulsorily handle the exception using
try catch or by throws keyword otherwise it would result in a compile time error.

```
Thread t=new Thread(); //new or born state
 t.start() // ready/runnable state
```

=> If T.S allocates cpu time then it would enter into running state.
=> If run() completes then it would enter a dead state.
=> If running thread invokes sleep(1000)/sleep(1000,100) then it would enter into Sleeping state
=> If time expires/ if sleeping thread got interrupted then thread would come back to "ready/runnable state".

synchronization

1. synchronized is a keyword applicable only for methods and blocks
2. if we declare a method/block as synchronized then at a time only one thread can execute that method/ block on that object.
3. The main advantage of synchronized keywords is we can resolve data inconsistency problems.
4. But the main disadvantage of synchronized keyword is it increases waiting time of the Thread and effects performance of the system.
5. Hence if there is no specific requirement then never recommended to use synchronized keyword.
6. Internally synchronization concept is implemented by using lock concept.
7. Every object in java has a unique lock. Whenever we are using synchronized keyword then only the lock concept will come into the picture.
8. If a Thread wants to execute any synchronized method on the given object 1st it has to get the lock of that object. Once a Thread gets the lock of that object then it's allowed to execute any synchronized method on that object. If the synchronized method execution completes then automatically Thread releases lock.
9. While a Thread executing any synchronized method the remaining Threads are not allowed to execute any synchronized method on that object simultaneously. But remaining Threads are allowed to execute any non-synchronized method simultaneously. [lock concept is implemented based on object but not based on method].

Note: Every object will have 2 area[Synchronized area and NonSynchronized area]
Synchronized Area => write the code only to perform update,insert,delete
NonSynchronized Area => write the code only to perform select operation

```
class ReservationApp{
 checkAvailablity(){
  //perform read operation
        }
 synchronized bookTicket(){
  //perform update operation
 }
}
```

class level lock

1. Every class in java has a unique level lock.
2. If a thread wants to execute static synchronized method then the thread requires "class level lock".
3. While a Thread executing any static synchronized method the remaining Threads are not allowed to execute any static synchronized method of that class simultaneously.
4. But remaining Threads are allowed to execute normal synchronized methods, normal static methods, and normal instance methods simultaneously.
5. Class level lock and object lock both are different and there is no relationship between these two.

synchronized block

```
synchronized void m1(){
 ...
 ...
 ...
 ...
 ...
 ======
 ======
 ======
 ======

 ...
 ...
 ...
 ...
 ...
}
```

if a few lines of code is required to get synchronized then it is not recommended to make the method only as synchronized.
If we do this then for threads performance will be low, to resolve this problem we use "synchronized block", due to synchronized block performance will be improved.

DeadLock

If 2 Threads are waiting for each other forever(without end) such type of situation (infinite waiting) is called dead lock.
There are no resolution techniques for deadlock but several prevention(avoidance) techniques are possible.
Synchronized keyword are the cause for deadlock hence whenever we are using synchronized keyword we have to take special care.

DeadLock vs starvation

- Long waiting of a thread, where waiting never ends is termed "deadlock".
- Long waiting for a thread, where waiting ends at a certain point is called "starvation".

eg:: Assume we have 1cr threads, where all 1cr threads have priority is 10,but one thread is there which has priority 0,now the thread with a priority-0 has to wait for long time but still
it gets a chance,but it has to wait for a long time, this scenario is called "Starvation".

Note: Low priority thread has to wait until completing all priority threads but ends at a certain point which is nothing but starvation.

Daemon Threads

The thread which is executing in the background is called "DaemonThread".
eg: AttachListener,SignalDispatcher,GarbageCollector,....

remember the example of movie
1. producer
2. director
3. music director
4. ....
5. ....
6. ....

MainObjective of DaemonThread

The main objective of DaemonThread, to provide support for Non-Daemon threads(main thread).
eg:: if main threads run with low memory then jvm will call GarbageCollector thread, to destroy the useless objects,so that no of bytes of free memory will be improved with this free memory the main thread can continue its execution.

Usually Daemon threads have low priority,but based on our requirement daemon threads can run with high priority also.
JVM => creates 2 threads
a. Daemon Thread(priority=1,priority=10)
b. main (priority=5)
while executing the main code, if there is a shortage of memory then immediately jvm will change the priority of Daemon thread to 10, so Garbage collector activates Daemon thread and it frees the memory after doing it immediately it changes the priority to 1, so the main thread will continue.

How to check whether the Thread is Daemon or not?

public boolean isDaemon() => To check whether the thread is "Daemon"
public void setDaemon(boolean b) throws IllegalThreadStateException
b=> true,means the thread will become Daemon,before starting the Thread we need to make the thread as "Daemon" otherwise it would result in "IllegalThreadStateException".

What is the default nature of the Thread?

Ans. By default the main thread is "NonDaemon ".for all remaining thread Daemon nature is inherited from Parent to child, that is If the parent thread is "Daemon" then the child thread will become "Daemon" and if the parent thread is "NonDaemon" then automatically the child thread is also "NonDaemon".

Is it possible to change the NonDameon nature of Main Thread?

Ans. Not possible, becoz the main thread starting is not in our hands, it will be started by "JVM".

How to stop a thread ?

As for how to start a method we have a method called t.start().
Similarly to stop a thread we have a method called t.stop().
This method will kill a thread and it makes the thread enter into a dead state.
This is not a good approach to kill a thread, so this method is "deprecated".

CompleteLifeCycle of a Thread

```
MyThread t=new MyThread();//"new or born state"
t.start();//"ready or runnable state"
```

a. if T.S allocates CPU time, then thread will be in "running state".
b. if running thread calls
    1. Thread.yield() it enters into ready/runnable state(it is just pausing)
    2. t2.join()
        t2.join(1000)
        t2.join(1000,100);//Thread enters into waiting state
            a. if t2 finishes the execution
            b. if time expires
            c. if thread gets interruption [it comes back to ready/runnable state]
    3. Thread.sleep(1000,100);
        Thread.sleep(100);//Thread enters into sleeping state
        a. if time expires
            b. if thread gets interruption [it comes back to ready/runnable state]
    4. obj.wait()
        obj.wait(1000);
        obj.wait(1000,100); //Thread enters into waiting state
            a. if thread gets notification
            b. if time expires
            c. if thread gets interruption [it comes back to ready/runnable state]
    5. t.suspend()
            a. it enters into suspended state
                a. if it calls t.resume()
                    it enters into ready/runnable state
    6. t.stop()
            a. it enters into dead state
b. if run() is completed by a thread, then the thread enters into "deadstate".

InterThreadCommunication

Two threads can communicate each other with the help of
a. notify()
b. notifyAll()
c. wait()

notify()=> Thread which is performing updates should call notify(),so the waiting thread will get notification so it will continue with its execution with the updated items.
wait() => Thread which is expecting notification/updation should call
wait(), immediately the Thread will enter into a waiting state.

wait(),notifyAll(),notify() is present in Object class, but not in Thread class why?

- Thread will call wait(),notify(),notifyall() on any type of objects like Student,Customer,Engineer.
- If a thread wants to call wait(),notify()/notifyall() then compulsorily the thread should be the owner of the object otherwise it would result in "IllegalMonitorStateException".
- We say thread to be owner of that object if thread has lock of that object.
- It means these methods are part of synchronized block or synchronized method, if we try to use outside the synchronized area then it would result in a RunTimeException called "IllegalMonitorStateException".
- if a thread calls wait() on any object, then first it immediately releases the lock on that object and it enters into waiting state.
- if a thread calls notify() on any object,then he may or may not release the lock on that object immediately.

Method prototype of wait(),notify(),notifyAll()

1. public final void wait()throws InterruptedException
2. public final native void wait(long ms)throws InterruptedException
3. public final void wait(long ms,int ns)throws InterruptedException
4. public final native void notify()
5. public final void notifyAll()

Difference b/w notify and notifyAll()

notify() => To give notification only for one waiting thread
notifyAll() => To give notification for many waiting thread

=> We can use notify() method to give notification for only one Thread. If multiple Threads are waiting then only one Thread will get the chance and remaining Threads have to wait for further notification. But which Thread will be notified(inform) we can't expect exactly it depends on JVM.
=> We can use notifyAll() method to give the notification for all waiting Threads of a particular object. All waiting Threads will be notified and will be executed one by one, because they require lock.

Note: On which object we are calling wait(), notify() and notifyAll() methods that corresponding object lock we have to get but not other object locks.